

Módulo 12 – Capítulo 09 – Sección 01

README técnico: descripción del sistema, arquitectura y cómo ejecutarlo localmente

La observabilidad del Capítulo 8 permite detectar y diagnosticar incidentes — pero solo si el ingeniero que recibe la alerta sabe qué sistema está operando, cómo está estructurado y qué comandos ejecutar para intervenir. El Capítulo 9 es el complemento humano del stack de observabilidad: los artefactos de documentación que convierten el conocimiento tácito del equipo fundador en conocimiento explícito accesible a cualquier ingeniero del equipo. El runbook operativo que cada alerta del Capítulo 8 referencia mediante un URL vive en este capítulo; el README que permite ejecutar el sistema localmente en menos de 30 minutos transforma el onboarding de semanas a días.

El README técnico del proyecto es el primer documento que lee un nuevo ingeniero y el último que escribe el equipo — lo que significa que frecuentemente está desactualizado en el momento más crítico: cuando el nuevo ingeniero lo necesita. La solución no es "actualizar el README después" — es tratar el README como parte del criterio de aceptación de cada PR que modifica el proceso de setup. Si el PR añade una nueva variable de entorno requerida, el PR no puede fusionarse sin actualizar `.env.example` y el paso correspondiente en el README.

El README incluye cinco secciones obligatorias que responden las cinco preguntas que cualquier nuevo ingeniero tiene en su primer día. La **descripción del sistema** (qué hace, para quién, en qué estado está) establece el contexto sin asumir que el lector conoce el proyecto. Dos párrafos concisos son suficientes: el primero describe el caso de uso y el usuario objetivo, el segundo describe el estado actual del sistema (versión, nivel de madurez, restricciones conocidas). La **arquitectura de alto nivel** usa un diagrama Mermaid renderizable en GitHub que muestra los componentes principales, los flujos de datos entre ellos y las dependencias externas (OpenAI, Cohere, Qdrant Cloud). El diagrama Mermaid es preferible a una imagen PNG porque se versiona en texto y puede actualizarse en el mismo commit que el código que cambia la arquitectura.

Los **prerrequisitos** listan las versiones exactas de cada herramienta necesaria para el setup local: Python 3.11.x (no "Python 3+"), Docker 24.x, Docker Compose 2.x, acceso a credenciales de OpenAI y Cohere (con instrucciones sobre cómo obtenerlos en el entorno de desarrollo), y acceso a Qdrant (instrucciones para el despliegue local con Docker o para usar Qdrant Cloud tier gratuito). La especificidad de versiones no es excesiva — es lo que previene que un ingeniero pase horas debuggeando un error causado por usar Python 3.9 en lugar de 3.11.

Las **instrucciones de setup local** son la sección más crítica del README y deben ser verificadas con un checkout fresco en una máquina limpia antes de cada release. El proceso en 7 pasos: `git clone`, `cd technical-assistant`, `cp .env.example .env` (y editar las credenciales reales), `docker compose up -d` (levanta Qdrant, Redis y PostgreSQL localmente), `pip install -e ".[dev]"` (instala la aplicación y las dependencias de desarrollo), `python scripts/create_test_collection.py` (crea la colección de Qdrant con datos de prueba), y `curl http://localhost:8000/health` para verificar que el sistema responde. Si este proceso funciona en menos de 30 minutos en una máquina limpia, el README cumple su función.

Secciones del README técnico

- **Descripción:** caso de uso (asistente técnico para ingenieros de software), usuario objetivo (equipos de ingeniería con documentación interna), estado del sistema (versión actual, restricciones de alcance documentadas, limitaciones conocidas).
- **Diagrama de arquitectura:** Mermaid graph LR con componentes (FastAPI, LangGraph, Qdrant, Redis, PostgreSQL) y flujos de datos; versionado en texto en el repositorio, actualizable en el mismo commit que el código.
- **Prerrequisitos:** Python 3.11.x, Docker 24.x, Docker Compose 2.x, API keys de OpenAI y Cohere (con instrucciones para obtenerlos), acceso a Qdrant (Docker local o Qdrant Cloud free tier).
- **Setup en 7 pasos:** verificado en máquina limpia antes de cada release; tiempo objetivo < 30 minutos; incluye comandos completos para copy-paste, no solo descripciones de alto nivel.
- **Troubleshooting:** los 3 errores más frecuentes en el setup local (basados en issues reportados por nuevos integrantes): `OPENAI_API_KEY not set`, puerto 8000 en uso (conflicto con otra aplicación) y error de conexión a Qdrant (Docker no corriendo).
- **Quick start:** un comando de verificación que confirma que el sistema está funcionando correctamente — `curl -X POST http://localhost:8000/query -H "... " -d '{"query":`

"¿qué es el pipeline RAG?")' con la respuesta esperada como referencia.

Nota del Arquitecto: La sección de Troubleshooting del README es la que más impacto tiene en el onboarding pero la que requiere más disciplina para mantener. Los errores más frecuentes en el setup local cambian con cada versión del sistema; si el Troubleshooting no se actualiza cuando cambian los errores comunes, los nuevos ingenieros siguen procedimientos que ya no aplican. La práctica que funciona es mantener un archivo `SETUP_ISSUES.md` donde cualquier ingeniero puede añadir un error que encontró y su solución, y revisar ese archivo antes de cada release para actualizar el Troubleshooting del README con los casos más recientes. Este proceso de curaduría distribuida produce un Troubleshooting mucho más completo que el que cualquier ingeniero individual podría mantener.

El README técnico no documenta el código — para eso existen los docstrings y los comentarios inline. Documenta el sistema: cómo desplegarlo, cómo verificar que funciona y cómo orientarse en el código para contribuir al componente correcto. El Capítulo 9 continúa con la documentación de API, que formaliza el contrato entre el sistema y sus consumidores.

Para recordar: Un README técnico que no permite a un nuevo ingeniero ejecutar el sistema localmente en menos de 30 minutos sin ayuda no está cumpliendo su función — debe actualizarse cada vez que cambia el proceso de setup.

Módulo 12 – Capítulo 09 – Sección 02

Documentación de API: endpoints, contratos y ejemplos de uso

La documentación de API es el contrato formal entre el sistema y sus consumidores. A diferencia del README —que documenta cómo ejecutar el sistema— la documentación de API documenta el comportamiento observable del sistema desde el exterior: qué endpoints existen, qué datos aceptan, qué datos retornan, y qué errores pueden producirse bajo qué condiciones. Para un sistema de IA con comportamiento no determinista como este asistente técnico, la documentación de API cumple una función adicional: establece qué aspectos del sistema son deterministas (la estructura del JSON de respuesta, los códigos de error, las validaciones de schema) y cuáles son variables (el texto de la respuesta, las fuentes citadas, el número de iteraciones del agente).

La elección de FastAPI como framework HTTP no fue arbitraria desde la perspectiva de la documentación. FastAPI genera automáticamente la especificación OpenAPI 3.1 a partir de los schemas Pydantic de los endpoints — lo que significa que la documentación de API es, por definición, consistente con el código que implementa la API. Cuando se añade un campo al schema `QueryRequest`, ese campo aparece automáticamente en la documentación de API en el próximo despliegue. Esto elimina la categoría de error más frecuente en la documentación de API de sistemas de software: la documentación que describe una versión anterior del sistema.

El endpoint principal `/query` es el corazón del contrato. Su schema de request `QueryRequest` acepta tres campos: `query` (string, obligatorio, longitud máxima 2000 caracteres), `session_id` (string opcional para habilitar la memoria de conversación del agente — si se omite, la interacción es stateless), y `filters` (objeto `QueryFilters` opcional con campos `document_types: list[str]`, `date_range: DateRange` y `team_attribution: str` para el filtrado RBAC a nivel de Qdrant). El schema de response `QueryResponse` retorna `answer` (string con la respuesta generada), `sources` (lista de objetos `Source` con `document_id`, `title`, `chunk_text` y `relevance_score`), y `metadata` (objeto `ResponseMetadata` con `agent_iterations`, `total_latency_ms`, `estimated_cost_usd` y `ragas_faithfulness` si la

petición fue evaluada online). Los tipos de datos de `estimated_cost_usd` y `ragas_faithfulness` son `Optional[float]` porque no están presentes en el 95% de las respuestas — solo en el 5% que cae en el sample de evaluación online.

Los códigos de error documentados no son los genéricos del framework — son los códigos que el sistema efectivamente retorna, con el mensaje de error correspondiente que permite al cliente diagnosticar el problema. El 400 incluye el JSON de `ValidationError` de Pydantic con el campo que falló la validación y el mensaje de error. El 401 incluye el motivo del rechazo del JWT (expirado, firma inválida, audience incorrecto). El 403 incluye el tipo de autorización fallida (RBAC — el usuario no tiene acceso al tipo de documento solicitado). El 429 incluye los headers `Retry-After` y `X-RateLimit-Remaining` para que el cliente pueda implementar backoff correcto. El 503 incluye el nombre de la dependencia que está indisponible (OpenAI API, Qdrant, Cohere) para facilitar el diagnóstico.

Elementos de la documentación de API

- **OpenAPI 3.1 spec autogenerada:** FastAPI genera la spec desde los schemas Pydantic; la documentación es garantizadamente consistente con el código en producción — no puede divergir.
- **Endpoint `/query`:** `QueryRequest` (query, session_id, filters) → `QueryResponse` (answer, sources, metadata con cost e iteraciones); tres ejemplos de request/response: pregunta simple, pregunta con filtros RBAC y pregunta fuera del dominio.
- **Endpoint `/ingest`:** `IngestRequest` (document_id, title, content, metadata con campos obligatorios: document_type, team, last_modified) y `BatchIngestRequest` (lista de hasta 100 documentos); response con `ingested_count`, `failed_ids` y `deduplication_stats`.
- **Códigos de error documentados:** 400 (`ValidationError` con campo y mensaje), 401 (JWT motivo de rechazo), 403 (RBAC tipo de acceso denegado), 429 (`Retry-After`, `X-RateLimit-Remaining`), 503 (dependencia indisponible con nombre).
- **Autenticación:** flujo JWT RS256 documentado con ejemplo de header `Authorization: Bearer <token>`, instrucciones para obtener el token del identity provider en entorno de desarrollo, y advertencia sobre la expiración de tokens (3600s por defecto, 900s en producción).
- **Entornos:** `/docs` (Swagger UI) y `/redoc` disponibles en desarrollo y staging; deshabilitados en producción con respuesta 404 para evitar exponer la spec a actores externos.

Nota del Arquitecto: La decisión de deshabilitar `/docs` y `/redoc` en producción es una medida de seguridad por defensa en profundidad, no un sustituto de la autenticación. Un atacante con acceso al código fuente puede reconstruir la spec de OpenAPI del código Pydantic — deshabilitar `/docs` solo previene que la spec esté disponible sin esfuerzo para quien ya tiene acceso al endpoint. El beneficio real es que un crawler genérico de APIs no puede descubrir la estructura del sistema; el impacto en ingenieros internos es nulo porque tienen acceso a la spec generada en el proceso de CI/CD y disponible en la documentación interna del equipo. La spec OpenAPI completa se publica en el repositorio interno de documentación como artefacto del pipeline CI/CD después de cada despliegue a producción.

La documentación de API no es el último paso del desarrollo de un endpoint — es parte de la definición de Done del endpoint. El criterio de aceptación de cada PR que añade o modifica un endpoint incluye verificar que los ejemplos de la documentación retornan las respuestas documentadas (test contract en el pipeline CI/CD) y que los códigos de error documentados son los que efectivamente retorna el endpoint en los casos de borde correspondientes.

Para recordar: La documentación de API no es un comentario — es un contrato entre el sistema y sus consumidores. Cuando la documentación está desincronizada con la implementación, no es el código el que está mal — es la documentación. El test contract en CI previene esta desincronización.

Módulo 12 – Capítulo 09 – Sección 03

Runbook operativo: procedimientos para los incidentes más frecuentes

El runbook operativo es el documento que transforma las alertas del Capítulo 8 de señales de diagnóstico en acciones de resolución. Cada alerta CRITICAL y WARNING del sistema tiene un campo `runbook_url` que apunta a la sección correspondiente del runbook — cuando el ingeniero de guardia recibe la alerta a las 3am, el primer paso no es pensar qué hacer: es abrir el runbook y seguir el procedimiento. Un runbook efectivo no asume conocimiento previo del sistema: cualquier ingeniero del equipo con acceso al cluster de Kubernetes y familiaridad general con los componentes debe poder seguirlo hasta resolución o hasta el punto de escalation.

La estructura de cada entrada del runbook no es opcional — la consistencia estructural es lo que permite al ingeniero de guardia orientarse bajo presión. Cada entrada incluye: **Alerta que lo activa** (nombre exacto de la alerta en Grafana y threshold), **Síntomas observables** (qué experimenta el usuario cuando ocurre el incidente, qué muestran los paneles del dashboard), **Diagnóstico** (comandos en orden de ejecución para confirmar la causa raíz), **Mitigación inmediata** (pasos con comandos completos, no descripciones de alto nivel — un ingeniero bajo presión no debe tener que inferir qué comando ejecutar), **Escalation** (cuándo y a quién escalar si la mitigación no produce el resultado esperado en 15 minutos), y **Cierre del incidente** (verificación cuantitativa de que el sistema volvió al estado normal y nota sobre si el incidente requiere post-mortem).

El primer incidente documentado — **Alta latencia P95 > 5000ms** — es el más frecuente en los primeros semanas post-despliegue. Los síntomas son latencia percibida por el usuario > 5s y el stat panel de P95 en rojo en el dashboard. El diagnóstico comienza con Grafana Tempo: abrir una traza reciente y verificar qué span consume la mayor proporción del tiempo total — el span `llm.generate` (indicativo de latencia del proveedor) o el span `rerank.cohere` (indicativo de latencia del reranker). Si el span `llm.generate` supera los 4000ms en la traza, el diagnóstico es latencia de OpenAI API: verificar con `curl https://api.openai.com/v1/models -H "Authorization: Bearer $OPENAI_API_KEY"` y revisar

el status de OpenAI en <https://status.openai.com>. Si la latencia de OpenAI es el problema, la mitigación es activar el modo degradado temporalmente: `kubectl set env deployment/assistant FALLBACK_MODE=true` — el sistema retorna respuestas de menor calidad usando únicamente el retrieval sin el agente, pero mantiene el SLA de latencia.

El segundo incidente — **Error rate > 5%** — requiere verificación inmediata de las dependencias críticas. El diagnóstico con Loki: `{app="assistant"} |= "ERROR" | json | line_format "{{.error}}"` para los últimos 5 minutos revela si el error es de tipo `openai.APIConnectionError`, `qdrant.ConnectionError` o `cohere.TooManyRequestsError`. Cada tipo de error tiene una mitigación diferente: para OpenAI connection errors, verificar el status de OpenAI y activar FALLBACK_MODE; para Qdrant connection errors, `kubectl exec -it qdrant-pod -- wget -q -O- http://localhost:6333/readyz` para verificar si el pod responde; para Cohere rate limit, reducir el sample de reranking temporalmente.

Incidentes documentados en el runbook

- **Alta latencia P95 > 5000ms:** diagnóstico con spans de Grafana Tempo (span más lento), comandos `kubectl top pods` para recursos, escalado horizontal con `kubectl scale deployment/assistant --replicas=5` si el cuello de botella es CPU; activación de FALLBACK_MODE si el problema es latencia del proveedor.
- **Error rate > 5%:** query Loki para tipo de error en los últimos 5 minutos, verificación de health de OpenAI/Qdrant/Cohere con comandos `curl` y `kubectl exec`, activación de circuit breaker y modo degradado si la dependencia está indisponible.
- **Degradación de faithfulness (Alerta WARNING):** comparación del faithfulness actual con el baseline en PostgreSQL (`SELECT * FROM evaluation_baselines ORDER BY created_at DESC LIMIT 5`), diagnóstico diferencial entre drift de datos (scores de reranking cayendo), drift de modelo (faithfulness cae en todas las categorías del golden dataset), y degradación de chunking (queries de alta complejidad tienen faithfulness bajo pero las simples no).
- **Pod OOM (OOMKilled):** `kubectl describe pod <pod-name>` para confirmar el evento OOMKilled, `kubectl logs <pod-name> --previous` para ver el stack trace del proceso antes del kill, ajuste de resource limits en el Helm chart como mitigación inmediata y análisis de memory profile con `py-spy` en staging para diagnóstico de raíz.
- **Rate limiting agresivo:** query Loki por `user_id` en los últimos 15 minutos para identificar el origen del volumen anómalo, bloqueo temporal del `user_id` en Redis con TTL de 1 hora si es abuso confirmado, investigación del bug del cliente si el volumen anómalo es de un usuario legítimo con comportamiento inesperado.

Nota del Arquitecto: *El runbook debe ser revisado por un ingeniero que no lo escribió antes de ser declarado listo para producción. La razón es simple: el ingeniero que escribe el runbook tiene conocimiento implícito del sistema que no se da cuenta de que está omitiendo. Cuando un ingeniero que no conoce el sistema lo lee y sigue los procedimientos, descubre invariablemente pasos que están incompletos o asumen contexto que no se explica. Esta revisión por un segundo ingeniero es la equivalente al "test de nuevo empleado" para la documentación de setup — si no puede seguirse sin ayuda adicional, necesita más detalle. El objetivo no es el runbook más completo posible — es el runbook que permite a un ingeniero con conocimiento general del sistema resolver el incidente más frecuente en menos de 15 minutos.*

Un runbook nunca está terminado — evoluciona con el sistema y con los incidentes reales. Cada incidente de producción que genere un post-mortem produce un aprendizaje: o el procedimiento de diagnóstico era incorrecto (hay que corregirlo), o faltaba una categoría de incidente (hay que añadirla), o el criterio de escalation era demasiado temprano o tardío (hay que ajustarlo). El runbook vivo es el que acumula ese aprendizaje.

Para recordar: Un runbook es efectivo cuando permite a un ingeniero con conocimiento general del sistema resolver el incidente sin ayuda del ingeniero que lo diseñó. Si el procedimiento asume conocimiento implícito, necesita más detalle — no menos.

Módulo 12 – Capítulo 09 – Sección 04

Documentación de seguridad: modelo de amenazas, controles y limitaciones conocidas

La documentación de seguridad es el artefacto que transforma el trabajo técnico del Capítulo 5 en conocimiento organizacional accesible. El Capítulo 5 implementó los controles; la documentación de seguridad los explica, los justifica, y —crucialmente— documenta sus limitaciones conocidas. Esta transparencia no es una debilidad que revelar a regañadientes: es el estándar de madurez de cualquier sistema en producción responsable. Los proyectos de software que ocultan sus limitaciones de seguridad no tienen menos vulnerabilidades que los que las documentan — tienen más, porque sus equipos no las abordan explícitamente.

La documentación de seguridad tiene un doble propósito. Externamente, permite a los equipos que consumen el sistema evaluar si su postura de riesgo es compatible con sus necesidades. Internamente, actúa como el contrato de seguridad del equipo consigo mismo: las decisiones sobre qué controlar y qué aceptar como riesgo residual se toman de forma explícita, no por omisión. El documento vive en el repositorio del proyecto (no en una wiki separada) y se actualiza en el mismo PR que añade o modifica controles de seguridad. Si el control cambia pero la documentación no se actualiza, la documentación describe un sistema que ya no existe.

La primera sección — el **modelo de amenazas** — presenta la tabla STRIDE aplicada al sistema con las ocho amenazas identificadas en el Capítulo 5, cada una con su probabilidad pre-control (Alta/Media/Baja), impacto potencial y el control que la mitiga. Las amenazas de mayor riesgo residual son las que más espacio reciben: el **prompt injection indirecto** (documentos en la base de conocimiento que contienen instrucciones dirigidas al LLM) y el **envenenamiento de la base de conocimiento** (documentos maliciosos insertados en el pipeline de ingesta). Para ambas, la tabla de amenazas documenta tanto el control implementado como la limitación: el classifier de intención detecta el 97% de los ataques de injection directa, pero su efectividad en idiomas fuera del inglés y el español cae al 78% — dato obtenido en el red teaming y documentado como riesgo residual explícito.

La segunda sección — **controles implementados** — lista cada control con: nombre del control, amenaza que mitiga, módulo Python donde está implementado con el nombre de la función o clase de referencia, y la efectividad medida en el red teaming del Capítulo 5. Esta sección no es una lista de características de seguridad para marketing interno — es una referencia técnica que permite a cualquier ingeniero del equipo localizar el código de implementación cuando el control produce un falso positivo o cuando necesita ajustarse.

La tercera sección — **limitaciones conocidas** — es la más importante del documento desde la perspectiva de la gestión de riesgos. Documenta tres categorías de limitaciones: limitaciones de los controles actuales (el intent classifier en idiomas minoritarios, la detección de PII que no cubre todos los formatos de datos sensibles), amenazas que el sistema no mitiga (ataques DoS de alto volumen sin WAF externo, compromiso del proveedor de LLM), y riesgos aceptados explícitamente con su justificación (el falso negativo del 2% en injection detection se acepta porque agregar un control adicional aumentaría el falso positivo rate de forma que degradaría la experiencia del usuario en el 3% de las queries legítimas).

Secciones de la documentación de seguridad

- **Threat model (tabla STRIDE):** ocho amenazas con probabilidad pre-control, impacto, control mitigante y estado del riesgo residual; prompt injection indirecto y envenenamiento de base de conocimiento como amenazas de mayor riesgo residual documentado.
- **Controles implementados:** tabla con nombre del control, amenaza mitigada, módulo Python de implementación (nombre de función/clase), efectividad medida en red teaming (% ataques mitigados por categoría) y fecha de última validación.
- **Limitaciones conocidas:** intent classifier en idiomas fuera de español/inglés (efectividad 78%), detección de PII incompleta en formatos no estándar, ausencia de WAF externo para DoS de alto volumen, dependencia del comportamiento del modelo del proveedor.
- **Proceso de security testing:** red teaming trimestral con OWASP LLM Top 10 como referencia, scope (los cinco vectores de ataque del sistema), herramientas (scripts custom + evaluación manual), y criterio de aprobación (tasa de bypass < 5% en todas las categorías).
- **Vulnerability reporting:** canal security@empresa.com con clave PGP pública, SLA de respuesta (24h inicial, 72h para triage de severidad, 7 días para plan de remediación), proceso de coordinated disclosure con ventana de 90 días antes de divulgación pública.

Nota del Arquitecto: Documentar las limitaciones de seguridad del sistema produce incomodidad en los equipos que no tienen práctica de seguridad madura — da la sensación de "admitir debilidades". La perspectiva correcta es la contraria: documentar las limitaciones conocidas es evidencia de un proceso de security testing riguroso que encontró las limitaciones, las evaluó cuantitativamente y tomó una decisión explícita sobre su tratamiento. Un sistema sin limitaciones documentadas no es un sistema sin vulnerabilidades — es un sistema sin un proceso de security testing que las hubiera encontrado. La transparencia sobre las limitaciones de seguridad es lo que permite a los equipos que consumen el sistema tomar decisiones informadas sobre qué cargas de trabajo son apropiadas para él.

La documentación de seguridad no es una fotografía del sistema en un momento — es un documento vivo que refleja el estado actual del sistema de seguridad. Cada resultado de red teaming, cada incidente de producción relacionado con seguridad, y cada cambio en los controles debe reflejarse en este documento. El proceso que garantiza esta actualización es el mismo que garantiza la actualización del README: la documentación de seguridad es parte del criterio de aceptación de cualquier PR que modifica los controles.

Para recordar: Documentar las limitaciones conocidas de seguridad no es un signo de debilidad — es el estándar de transparencia de los sistemas en producción responsables. La diferencia entre un sistema seguro y un sistema que parece seguro es exactamente ese nivel de honestidad sobre las limitaciones reales.

Módulo 12 – Capítulo 09 – Sección 05

Guía de contribución: cómo extender el sistema y añadir nuevas capacidades

El sistema integrador que el equipo fundador construyó a lo largo de los Capítulos 1 al 8 no es el sistema que operará en seis meses. Los requisitos de los usuarios evolucionan, nuevas fuentes de documentación se añaden, el equipo incorpora ingenieros que no estuvieron en el diseño inicial, y las capacidades del ecosistema (nuevos modelos, nuevas herramientas de observabilidad, nuevos controles de seguridad) abren posibilidades que el diseño original no podía anticipar. La guía de contribución es el puente entre el sistema que existe y el sistema que el equipo necesita que sea — el documento que permite a un nuevo ingeniero añadir una capacidad al sistema sin romper las existentes, sin duplicar código que ya existe, y sin introducir patrones de diseño inconsistentes con los que el equipo eligió.

La guía de contribución no documenta el sistema en general — eso lo hace el README. Documenta los patrones de extensión específicos: las interfaces que el sistema expone para ser extendido, los puntos de integración donde añadir nueva funcionalidad, y los tests que toda extensión debe pasar antes de fusionarse. La diferencia es crucial: una guía de contribución que explica "cómo funciona el sistema" es un sustituto del README; una que explica "cómo añadir una nueva herramienta al agente" es lo que permite a un ingeniero hacer esa extensión en medio día en lugar de dos días de arqueología del código.

Las tres extensiones más comunes están documentadas con procedimientos paso a paso. La primera — **añadir una nueva herramienta al agente** — requiere seguir cuatro pasos en orden. Primero, implementar la función con el decorador `@tool` de LangChain: el docstring de la función es lo que el LLM lee para decidir cuándo usar la herramienta, por lo que debe describir claramente el propósito de la herramienta y en qué situaciones es apropiada usarla (no en qué situaciones no usarla — el LLM infiere las negativas). Segundo, definir el schema de inputs como clase Pydantic con `Field(description="...")` en cada campo — la descripción de cada campo es igualmente parte del contrato que el LLM usa para construir la llamada correcta. Tercero, registrar la herramienta en la lista `tools` del grafo LangGraph en `graph.py`. Cuarto, añadir tests unitarios que mocken el resultado de la herramienta y

verifiquen el comportamiento del agente ante respuestas correctas, respuestas vacías y excepciones.

La segunda extensión — **añadir una nueva fuente de datos al pipeline de ingesta** — requiere implementar la interfaz `BaseDocumentParser` con el método `parse(self, source: str) -> list[Document]`. La implementación debe retornar objetos `Document` con los campos de metadata obligatorios (`document_id`, `title`, `document_type`, `team`, `last_modified`) y opcionales (`author`, `url`, `version`). Una vez implementado el parser, debe registrarse en la fábrica de conectores `ParserFactory` añadiendo su mapping en el diccionario `PARSER_REGISTRY`. Los tests del parser usan fixtures de documentos de ejemplo que cubren los formatos más comunes del tipo de fuente.

La tercera extensión — **cambiar el modelo LLM** — es la que tiene más impacto sistémico y la que más rigor requiere. El proceso obliga a: ejecutar el golden dataset de evaluación completo con el nuevo modelo antes de modificar ningún código de producción, comparar `faithfulness` y `answer_relevance` con el baseline del modelo actual, actualizar ADR-001 con la nueva decisión y su justificación basada en los resultados de evaluación, y actualizar `.env.example` con el nuevo nombre del modelo. Si el cambio es de familia de modelos (OpenAI → Anthropic), el proceso añade la validación del sistema prompt con el nuevo modelo (los system prompts optimizados para GPT-4o no son necesariamente óptimos para Claude 3.5 Sonnet).

Extensiones documentadas en la guía de contribución

- **Nueva herramienta del agente:** decorador `@tool`, docstring descriptivo para el LLM, schema Pydantic con `Field(description)`, registro en `tools` de LangGraph, tests unitarios con mock de resultado y fallo, actualización del system prompt si la herramienta requiere instrucciones específicas de uso.
- **Nueva fuente de ingesta:** implementar `BaseDocumentParser.parse()`, campos de metadata obligatorios y opcionales, registro en `ParserFactory.PARSER_REGISTRY`, tests con fixtures de documentos de ejemplo del tipo de fuente.
- **Cambio de modelo LLM:** evaluación con golden dataset antes de cualquier código, comparación con baseline, actualización de ADR-001 con justificación basada en datos, ajuste del system prompt al comportamiento del nuevo modelo.
- **Cambio de modelo de embedding:** requiere re-indexación completa de Qdrant (el espacio vectorial cambia); proceso blue-green documentado paso a paso con nueva

colección en paralelo, evaluación de retrieval con el golden dataset, cutover con downtime cero.

- **Proceso de PR:** checklist de tests (unitarios, integración, contrato de API), criterios de code review (cobertura > 80% en código nuevo, sin secrets, sin CVEs críticos nuevos en Trivy), quién debe aprobar (1 lead del área + 1 revisor de seguridad para cambios en Cap 05) y tiempo esperado de respuesta (48h en días hábiles).

Nota del Arquitecto: La documentación de la extensión "cambio de modelo de embedding" merece atención especial porque es la extensión con mayor riesgo operativo de las cinco. Un cambio de modelo de embedding invalida todos los vectores almacenados en Qdrant — los vectores producidos por `text-embedding-3-small` no son comparables con los producidos por `text-embedding-ada-002`, y mezclar vectores de distintos modelos en la misma colección produce resultados de retrieval sin sentido. El proceso blue-green que la guía documenta es el único que garantiza continuidad del servicio durante la migración: crear una segunda colección en Qdrant con el nuevo modelo, re-indexar todos los documentos en la nueva colección mientras la antigua sigue sirviendo tráfico, validar el retrieval de la nueva colección con el golden dataset, y hacer el cutover actualizando la variable de entorno `QDRANT_COLLECTION_NAME` en el ConfigMap. Si el proceso falla en cualquier punto, el rollback es trivial: la colección antigua sigue intacta.

La guía de contribución es un documento que se escribe por primera vez con el sistema inicial y que se enriquece con cada extensión real que el equipo implementa. La primera vez que un nuevo ingeniero añade una herramienta al agente siguiendo la guía, la experiencia de ese proceso revela exactamente qué pasos estaban incompletos o asumían contexto no documentado — esas correcciones deben incorporarse a la guía en el mismo PR. Una guía de contribución que evoluciona con el sistema refleja la madurez del equipo; una que permanece igual que en la versión inicial refleja que nadie la está usando.

Para recordar: Una guía de contribución que documenta las extensiones más comunes del sistema reduce el tiempo de onboarding de semanas a días, y convierte el conocimiento tácito del equipo fundador en un activo organizacional que persiste independientemente de la rotación del equipo.

Módulo 12 – Capítulo 09 – Sección 06

Cierre: un sistema bien documentado puede ser operado y evolucionado por cualquier ingeniero del equipo

El Capítulo 9 completó la última capa del sistema integrador — la que hace posible que el sistema sobreviva más allá de los ingenieros que lo construyeron. Los sistemas de software no fallan únicamente por bugs en el código o degradaciones del modelo; fallan porque el ingeniero que llega a las 3am a resolver un incidente no sabe qué hacer con el sistema que recibe, porque el nuevo integrante del equipo que necesita añadir una fuente de datos no puede encontrar el punto de extensión correcto, porque el equipo de seguridad que audita el sistema no puede determinar qué controles están activos y cuáles son sus limitaciones. La documentación técnica es la respuesta a esas tres situaciones, y su ausencia o desactualización produce los mismos fallos de operación que los bugs de producción — solo que con una causa más difícil de diagnosticar y más costosa de remediar.

Los cinco artefactos del Capítulo 9 no son independientes — forman un sistema de conocimiento cohesivo que cubre el ciclo completo del sistema. El README permite al nuevo ingeniero llegar al sistema. La documentación de API le permite usarlo como consumidor. El runbook le permite operarlo cuando algo falla. La documentación de seguridad le permite entender sus garantías y sus limitaciones. Y la guía de contribución le permite extenderlo con nuevas capacidades. Cada artefacto responde a una pregunta diferente del ingeniero en un momento diferente de su relación con el sistema — juntos, eliminan la dependencia de un conocimiento que solo vive en la cabeza del ingeniero fundador.

La práctica que hace sostenible esta documentación no es dedicar una semana al final del proyecto a escribirla — es tratarla como un ciudadano de primera clase del repositorio que se actualiza en el mismo commit que el código que cambia. Cuando el pipeline de ingesta añade soporte para un nuevo tipo de fuente, el PR incluye: el código del nuevo parser, los tests, y la sección correspondiente en la guía de contribución. Cuando el red teaming trimestral encuentra una nueva categoría de ataque, el resultado se incorpora en la documentación de seguridad antes de cerrar el ciclo. Cuando el runbook guía un incidente real y el procedimiento resulta incorrecto, la corrección se documenta en el mismo post-mortem. Esta

disciplina de documentación como práctica de ingeniería no es overhead — es la inversión que hace que el próximo incidente, la próxima incorporación y la próxima extensión cuesten menos que la anterior.

Artefactos del Capítulo 9

- **README técnico:** descripción del sistema, diagrama Mermaid de arquitectura, prerequisites con versiones exactas, setup en 7 pasos verificado en máquina limpia, troubleshooting con los 3 errores más frecuentes. Objetivo: un nuevo ingeniero puede ejecutar el sistema localmente en menos de 30 minutos.
- **Documentación de API:** OpenAPI 3.1 autogenerada desde Pydantic, enriquecida con ejemplos de request/response para los casos principales y de borde, códigos de error documentados con mensajes concretos, flujo de autenticación JWT explicado.
- **Runbook operativo:** cinco incidentes frecuentes con estructura consistente (síntomas, diagnóstico, mitigación, escalation, cierre). Criterio de efectividad: un ingeniero con conocimiento general del sistema puede resolver el incidente en menos de 15 minutos.
- **Documentación de seguridad:** threat model STRIDE con controles y limitaciones conocidas, proceso de security testing trimestral, canal de vulnerability reporting con SLA. La sección de limitaciones es la más importante — documenta la postura de riesgo aceptada explícitamente.
- **Guía de contribución:** tres extensiones comunes documentadas paso a paso (nueva herramienta, nueva fuente de datos, cambio de modelo), proceso de PR con checklist y criterios de aceptación, tiempo esperado de respuesta.

Nota del Arquitecto: Hay una métrica informal pero muy útil para evaluar la calidad de la documentación técnica de un sistema: el "tiempo de primera contribución" — cuánto tiempo le toma a un nuevo ingeniero hacer su primera contribución al repositorio sin ayuda del equipo. En proyectos con documentación de calidad, ese tiempo es de 1-3 días; en proyectos sin documentación o con documentación desactualizada, es de 1-3 semanas. El costo de esa diferencia es fácil de calcular con el salario del ingeniero — pero el costo real incluye también la frustración del nuevo integrante, la interrupción del equipo fundador al que pide ayuda repetidamente, y la calidad del código de esa primera contribución cuando se hace con conocimiento incompleto del sistema. Una semana de ingeniería invertida en documentación de calidad tiene un ROI que se recupera en el primer onboarding.

Un sistema que puede operarse y evolucionarse por cualquier ingeniero del equipo no es solo más fácil de mantener — es más resiliente. La concentración de conocimiento en una persona es el punto único de falla más peligroso de cualquier sistema de software. La documentación técnica distribuye ese conocimiento, y su mantenimiento como práctica de equipo es la garantía de que el sistema puede sobrevivir a la rotación, al crecimiento y al tiempo.

"The ratio of time spent reading versus writing is well over 10 to 1. We are constantly reading old code as part of the effort to write new code. Making it easy to read makes it easier to write." — Robert C. Martin, Clean Code